```
1  ----------------------------------------------------------------------------------------------------
2  -- 16-bit Q1.14 Cordic Calculator Testbench
3  --
4  --  This entity implements a test bench for a 16-bit Q1.14 Cordic calculator.
5  --  Test values are compared against standard library functions
6
7  --  Revision History:
8  --    Date:         Author:    Description:
9  --    07 Jan 2024   Chris M.   Initial revision; described entity and processes.
10 --    18 Feb 2025   Chris M.   Converted to OSVVM.
11 --    21 Feb 2025   Chris M.   Restructured tests to include function type in bin
12 --    22 Feb 2025   Chris M.   Modified test bench for pipelined design.
13 --    25 Feb 2025   Chris M.   Modified test bench to allow for 3 pipeline stages.
14 --
15 --  TODO:
16 --    - Decrease error in hyperbolic functions.
17 --    - Make tests actually random (maybe make all bins pick X and Y and then repick based on the
18 --      function to make sure it won't overflow...)
19 --    - Remove magic numbers.
20 ----------------------------------------------------------------------------------------------------
21
22 -- libraries
23 library ieee;
24 library std;
25 library osvvm;
26 library work;
27
28 use ieee.std_logic_1164.all;      -- For std_logic operations
29 use ieee.std_logic_unsigned.all;  -- For unsigned operations
30 use ieee.numeric_std.all;         --
31 use ieee.std_logic_textio.all;    -- For std_logic IO
32 use std.textio.all;               -- For text IO
33 use std.env.all;
34 use ieee.math_real.all;           -- For trig and arithmetic functions.
35
36 use osvvm.RandomPkg.all;
37 use osvvm.CoveragePkg.all;
38 use osvvm.TranscriptPkg.all ;
39 context osvvm.OsvvmContext;
40
41 use work.CordicConstants.all;     -- Cordic constants.
42 use work.FixedLib.all;            -- Fixed point conversion and printing libs
43 use work.all;
44 use ANSIEscape.all;               -- ANSI Escape sequences for printing color.
45
46 entity Cordic_TB is
47 end Cordic_TB;
48
49 architecture TestBench of Cordic_TB is
50
51   constant WORD_SIZE : positive := 16;
52
53   signal ClkEn    : std_logic := '1';
54   signal Clk_Tb   : std_logic;
```

```
55  signal X_Tb     : std_logic_vector(15 downto 0);
56  signal Y_Tb     : std_logic_vector(15 downto 0);
57  signal Func_Tb  : std_logic_vector(4  downto 0);
58  signal R_Tb     : std_logic_vector(15 downto 0);
59
60
61  -- Convert std_logic_vectors to unsigned ints
62  function slv_to_uint(slv : std_logic_vector) return integer is
63  begin
64    return to_integer(unsigned(slv));
65  end function;
66
67  -- Repeat a char n times and return a string.
68  function RepeatChar(c : character; n : positive) return string is
69    variable Result : string(1 to n) := (others => ' ');
70  begin
71    for i in 1 to n loop
72      Result(i) := c;
73    end loop;
74    return Result;
75  end function;
76
77  -- Convert signed ints to std_logic_vectors.
78  function int_to_slv(i : integer; width : integer) return std_logic_vector is
79  begin
80    return std_logic_vector(to_signed(i, width));
81  end function;
82
83  -- Convert std_logic_vectors to signed ints.
84  function slv_to_int(slv : std_logic_vector) return integer is
85  begin
86    return to_integer(signed(slv));
87  end function;
88
89  -- Convert unsigned ints to std_logic_vectors.
90  function uint_to_slv(i : natural; width : positive) return std_logic_vector is
91  begin
92    return std_logic_vector(to_unsigned(i, width));
93  end function;
94
95  -- Convert std_logic to bit. (Unusued.)
96  -- function ToBit(sl : std_logic) return bit is
97  -- begin
98  --   if ((sl /= '1') and (sl /= '0')) then
99  --     return '0';
100 --   elsif (sl = '0') then
101 --     return '0';
102 --   else
103 --     return '1';
104 --   end if;
105 -- end function;
106
107 -- Convert bit to std_logic. (Unused.)
108 -- function ToStdLogic(b : bit) return std_logic is
```

```
109  -- begin
110  --   if (b = '0') then
111  --     return '0';
112  --   else
113  --     return '1';
114  --   end if;
115  -- end function;
116
117  -- Period for a 1GHz clock
118  constant CLK_PERIOD : time  := (1 sec / (1e9));
119
120  -- The min and max Q1.14 fixed point values represented as reals.
121  constant MIN_REAL_VAL : real := -2.0;
122  constant MAX_REAL_VAL : real := 1.9999;
123
124  -- The min and max angles such that trig functions converge represented as reals.
125  constant MIN_REAL_ANGLE : real := 0.0;
126  constant MAX_REAL_ANGLE : real := MATH_PI / 2.0;
127
128  -- The min and max Q1.14 fixed point values represented as Q1.14 fixed points.
129  constant MIN_FIXED_VAL : std_logic_vector(WORD_SIZE - 1 downto 0) := RealToQ1_14(MIN_REAL_VAL);
130  constant MAX_FIXED_VAL : std_logic_vector(WORD_SIZE - 1 downto 0) := RealToQ1_14(MAX_REAL_VAL);
131
132  -- The min and max angles such that trig functions converge represented as Q1.14 fixed points.
133  constant MIN_FIXED_ANGLE : std_logic_vector(WORD_SIZE - 1 downto 0) := RealToQ1_14(MIN_REAL_ANGLE);
134  constant MAX_FIXED_ANGLE : std_logic_vector(WORD_SIZE - 1 downto 0) := RealToQ1_14(MAX_REAL_ANGLE);
135
136  -- Hyperbolic functions do not converge for x > +- 1.11 (the sum of the hyperbolic constansts)
137  -- see Walther, 1971. Ours converges within the minimum preicsion for x <= +-1.09 (why?)
138  constant MAX_HYPERBOLIC_ARG : std_logic_vector(WORD_SIZE - 1 downto 0) := RealToQ1_14(1.06);
139  constant MIN_HYPERBOLIC_ARG : std_logic_vector(WORD_SIZE - 1 downto 0) := RealToQ1_14(-1.06);
140
141  -- Integer and fractional bits for a Q1.14 fixed point.
142  constant INTEGER_BITS    : positive := 2;
143  constant FRACTIONAL_BITS : positive := 14;
144
145  -- The precision of a Q1.14 fixed point.
146  constant PRECISION : real := (1.0 / (2.0 ** FRACTIONAL_BITS));
147
148  -- Maybe ?
149  constant LATENCY : integer := 3;
150
151  -- Checks if a a function will overflow (ie, be more than the range representable by a Q1.14
152  -- fixed point).
153  function WillOverflow(X : real; Y : real;
154                        Func : std_logic_vector) return boolean is
155  begin
156    case (Func) is
157      when F_COS_X =>
158        if ((cos(x) > MAX_REAL_VAL) or (cos(x) < MIN_REAL_VAL)) then
159          return true;
160        end if;
161      when F_SIN_X =>
162        if ((sin(x) > MAX_REAL_VAL) or (sin(x) < MIN_REAL_VAL)) then
```

```
163           return true;
164         end if;
165       when F_X_MULT_Y =>
166         if (((x * y) > MAX_REAL_VAL) or ((x * y) < MIN_REAL_VAL)) then
167           return true;
168         end if;
169       when F_COSH_X =>
170         if ((cosh(x) > MAX_REAL_VAL) or (cosh(x) < MIN_REAL_VAL)) then
171           return true;
172         end if;
173       when F_SINH_X =>
174         if ((sinh(x) > MAX_REAL_VAL) or (sinh(x) < MIN_REAL_VAL)) then
175           return true;
176         end if;
177       when F_Y_DIV_X =>
178         if (x = 0.0) then
179           -- Anything divided by zero is undefined
180           return true;
181         elsif ( ((y / x) > MAX_REAL_VAL) or ( (y/x) < MIN_REAL_VAL)) then
182           return true;
183         end if;
184       when others =>
185         report "Unknown operation"
186         severity ERROR;
187     end case;
188     return false;
189   end function;
190 
191   constant ZERO : std_logic_vector(WORD_SIZE - 1 downto 0) := (others => '0');
192 
193   constant SPECIAL_CASE   : integer := RealToQ1_14(0.75);
194   constant SPECIAL_CASE_1 : integer := RealToQ1_14(0.25);
195 
196   -- Bins for the function select lines.
197   constant TRIG_FUNC_BIN : CovBinType :=
198     GenBin(slv_to_uint(F_COS_X)) & GenBin(slv_to_uint(F_SIN_X));
199 
200   constant LINEAR_FUNC_BIN : CovBinType :=
201     GenBin(slv_to_uint(F_X_MULT_Y)) & GenBin(slv_to_uint(F_Y_DIV_X));
202 
203   constant HYPERBOLIC_FUNC_BIN : CovBinType :=
204     GenBin(slv_to_uint(F_COSH_X)) & GenBin(slv_to_uint(F_SINH_X));
205 
206   -- Bins for trig function inputs. The test strategy for trig functions is to have 7 bins total.
207   -- 4 bins consist of the range of valid inputs for trig functions, divided into 4 sections, and
208   -- the other three are likely edge cases (min, max, and zero).
209   constant TRIG_X_BIN : CovBinType :=
210     GenBin(slv_to_int(MIN_FIXED_ANGLE), slv_to_int(MAX_FIXED_ANGLE), 4) &
211     GenBin(slv_to_int(MIN_FIXED_ANGLE)) & GenBin(slv_to_int(MAX_FIXED_ANGLE)) &
212     GenBin(slv_to_int(ZERO));
213 
214   -- Bins for hyperbolic functions. The test strategy is the same as trig functions but with
215   -- different upper/lower limits.
216   constant HYPER_X_BIN : CovBinType :=
```

```
217     GenBin(slv_to_int(MIN_HYPERBOLIC_ARG), slv_to_int(MAX_HYPERBOLIC_ARG), 4) &
218     GenBin(slv_to_int(MIN_HYPERBOLIC_ARG)) & GenBin(slv_to_int(MAX_HYPERBOLIC_ARG)) &
219     GenBin(slv_to_int(ZERO));
220
221   -- Bins for linear functions. They consists of the range of our fixed points divided into 4
222   -- sections. Note that we do not include bins for max and min values because cross binning would
223   -- generate bins that will overflow and thus never be covered (such as 2.0 * 2.0, etc).
224   constant LINEAR_X_BIN : CovBinType :=
225     GenBin(slv_to_int(MIN_FIXED_VAL), slv_to_int(MAX_FIXED_VAL), 4);
226
227   constant LINEAR_Y_BIN : CovBinType :=
228     GenBin(slv_to_int(MIN_FIXED_VAL), slv_to_int(MAX_FIXED_VAL), 4);
229
230   -- Coverage IDs
231   signal CordicTrigCovID   : CoverageIdType := NewId("Trig Cordic Coverage");
232   signal CordicLinearCovID : CoverageIdType := NewId("Linear Cordic Coverage");
233   signal CordicHyperCovID  : CoverageIdType := NewId("Hyperbolic Cordic Coverage");
234
235   -- Log ID
236   signal CordicTBLogID       : AlertLogIDType := GetAlertLogID("Cordic_TB");
237
238
239   -- A 5-tuple of 7 char strings
240   type String5Tup is array(0 to 4) of string(1 to 7);
241
242   -- subtype UnsignedWordInt is integer range 0 to 2**WORD_SIZE - 1;
243   subtype SignedWordInt   is integer range -(2 ** (WORD_SIZE - 1)) to (2 ** (WORD_SIZE - 1)) - 1;
244
245   -- A 3-tuple of integers
246   type Int5Tup is array(0 to 4) of SignedWordInt;
247
248   -- Print the current function to the debug stream
249   procedure PrintFunc(X : integer; Y : SignedWordInt; Func : SignedWordInt) is
250   begin
251     case int_to_slv(Func, Func_TB'length) is
252       when F_COS_X =>
253         Log("Picked:" & HT & "cos(" & to_string(Q1_14ToReal(X)) & ")", DEBUG);
254       when F_SIN_X =>
255         Log("Picked:" & HT & "sin(" & to_string(Q1_14ToReal(X)) & ")", DEBUG);
256       when F_COSH_X =>
257         Log("Picked:" & HT & "cosh(" & to_string(Q1_14ToReal(X)) & ")", DEBUG);
258       when F_SINH_X =>
259         Log("Picked:" & HT & "sinh(" & to_string(Q1_14ToReal(X)) & ")", DEBUG);
260       when F_X_MULT_Y =>
261         LOG("Picked:" & HT & to_string(Q1_14ToReal(X)) & " * " &
262                   to_string(Q1_14ToReal(Y)), DEBUG);
263       when F_Y_DIV_X =>
264         Log("Picked:" & HT & to_string(Q1_14ToReal(Y)) & " / " &
265                   to_string(Q1_14ToReal(X)), DEBUG);
266       when others =>
267         Log("Unknown function: " & to_string(Func), DEBUG);
268     end case;
269   end procedure;
270
```

```
271 begin
272
273   -- Wire the entity
274   UUT : entity Cordic
275   port map (
276     CLK   => CLK_TB,
277     X     => X_TB,
278     Y     => Y_TB,
279     Func  => Func_TB,
280     R     => R_TB
281   );
282
283   -- Generate the clock
284   GenClk : process
285   begin
286     if    (ClkEn = '1') then
287       Clk_TB <= '0';
288       wait for CLK_PERIOD / 2;
289       Clk_TB <= '1';
290       wait for CLK_PERIOD / 2;
291     else
292       wait until ClkEn = '1';
293     end if;
294   end process;
295
296   SetTestName("CORDIC Tests");
297
298   SetLogEnable(PASSED, true);
299   SetLogEnable(DEBUG,  false);
300   SetLogEnable(INFO,   false);
301
302   -- AddCross(<ID : CoverageIdType>, <CovGoal : integer>, <Bins : CovBinType>, ... );
303
304   -- Cross the bins.
305   AddCross(CordicTrigCovID,   4, TRIG_FUNC_BIN,       TRIG_X_BIN);
306   AddCross(CordicLinearCovID, 4, LINEAR_FUNC_BIN,     LINEAR_X_BIN, LINEAR_Y_BIN);
307   AddCross(CordicHyperCovID,  4, HYPERBOLIC_FUNC_BIN, HYPER_X_BIN);
308
309   RunTest : process(Clk_TB)
310
311     -- The random variable
312     variable RV : RandomPType;
313
314     -- The current Function, X, and Y as (unsigned) integers
315     variable CurrentFunc : SignedWordInt;
316     variable CurrentX    : SignedWordInt;
317     variable CurrentY    : SignedWordInt;
318
319     -- The expected result as an unsigned integer.
320     variable ExpectedResult    : SignedWordInt := 0;
321
322     -- The current function as a string.
323     variable CurrentFuncString : string(1 to 7) := (others => ' ');
324
```

```
325     -- The past 5 functions, X, and Y as unsigned integers.
326     variable PrevFuncSync : Int5Tup := ( 0, 0, 0 ,0, 0) ;
327     variable PrevXSync    : Int5Tup := ( 0, 0, 0 ,0, 0);
328     variable PrevYSync    : Int5Tup := ( 0, 0, 0 ,0, 0);
329
330     -- The past 5 functions as strings.
331     variable PrevFuncStringSync     : String5Tup :=
332       ( (others => ' '), (others => ' '), (others => ' '), (others => ' '), (others => ' ') );
333
334     -- The past 5 results as unsigned integers.
335     variable PrevExpectedResultSync : Int5Tup := ( 0, 0, 0, 0, 0 );
336
337     -- Check if the RV has been seeded
338     variable Seeded : boolean := false;
339
340     -- The number of rising edges that have occurred.
341     variable EdgeCount : integer := 0;
342
343     variable Cond : boolean;
344     variable Color : string(1 to 5) := (others => ' ');
345
346     variable TmpFunc : SignedWordInt := 0;
347     variable TmpX    : SignedWordInt := 0;
348     variable TmpY    : SignedWordInt := 0;
349
350     variable Diff : real := 0.0;
351
352     variable TrigDone   : boolean := false;
353     variable LinearDone : boolean := false;
354     variable HyperDone  : boolean := false;
355
356     -- variable test_int  : integer := 0;
357     -- variable test_real : real    := 0.0;
358
359   begin
360
361     if (not Seeded) then
362
363       Log("Seeding Random Variable ..." ,  ALWAYS);
364       Log("Min Fixed Val is      " & to_string(Q1_14ToReal(MIN_FIXED_VAL)) &
365           " (" & to_string(slv_to_int(MIN_FIXED_VAL)) & ")", ALWAYS);
366       Log("Max Fixed Val is      " & to_string(Q1_14ToReal(MAX_FIXED_VAL)) &
367           " (" & to_string(slv_to_int(MAX_FIXED_VAL)) & ")", ALWAYS);
368       Log("Min Fixed Angle is    " & to_string(Q1_14ToReal(MIN_FIXED_ANGLE)) &
369           " (" & to_string(slv_to_int(MIN_FIXED_ANGLE)) & ")", ALWAYS);
370       Log("Max Fixed Angle is    " & to_string(Q1_14ToReal(MAX_FIXED_ANGLE)) &
371           " (" & to_string(slv_to_int(MAX_FIXED_ANGLE)) & ")", ALWAYS);
372       Log("Min Hyperbolic Arg is " & to_string(Q1_14ToReal(MIN_HYPERBOLIC_ARG)) &
373           " (" & to_string(slv_to_int(MIN_HYPERBOLIC_ARG)) & ")", ALWAYS);
374       Log("Max Hyperbolic Arg is " & to_string(Q1_14ToReal(MAX_HYPERBOLIC_ARG)) &
375           " (" & to_string(slv_to_int(MAX_HYPERBOLIC_ARG)) & ")", ALWAYS);
376
377       -- Seed the random variable
378       RV.InitSeed(RV'instance_name);
```

```
379       Seeded := true;
380
381       if  (IsLogEnabled(DEBUG))  then Log("Logging Enabled", DEBUG);  end if;
382       if  (IsLogEnabled(PASSED)) then Log("Logging Enabled", PASSED); end if;
383
384     elsif (falling_edge(Clk_TB)) then
385
386       Log(YELLOW & "Falling Edge" & ANSI_RESET, DEBUG);
387
388       -- Skip the first 5 checks while we wait for the results to come in.
389       if (EdgeCount < 5) then
390         EdgeCount  := EdgeCount + 1;
391         Log("Skipping...", DEBUG);
392       else
393
394         -- The test has passed if the result is within the precision allowed by a Q1.14 fixed point
395         Diff  := Q1_14ToReal(R_TB) - Q1_14ToReal(PrevExpectedResultSync(LATENCY));
396         Cond  := Diff <= PRECISION;
397
398         -- Highlight green if passes, red if failed.
399         if (Cond) then
400           Color := GREEN;
401         else
402           Color := RED;
403         end if;
404
405         -- Log prev functions and inputs.
406         Log(to_string(LF), DEBUG);
407         Log(          "PrevFuncStringSync(4): " & PrevFuncStringSync(4) , DEBUG);
408         Log(          "PrevFuncStringSync(3): " & PrevFuncStringSync(3) , DEBUG);
409         Log(          "PrevFuncStringSync(2): " & PrevFuncStringSync(2) , DEBUG);
410         Log(          "PrevFuncStringSync(1): " & PrevFuncStringSync(1) , DEBUG);
411         Log(MAGENTA & "PrevFuncStringSync(0): " & PrevFuncStringSync(0) & ANSI_RESET, DEBUG);
412         Log(to_string(LF), DEBUG);
413         Log(          "PrevXSync(4): "           & to_string(Q1_14ToReal(PrevXSync(4))) , DEBUG);
414         Log(          "PrevXSync(3): "           & to_string(Q1_14ToReal(PrevXSync(3))) , DEBUG);
415         Log(          "PrevXSync(2): "           & to_string(Q1_14ToReal(PrevXSync(2))) , DEBUG);
416         Log(          "PrevXSync(1): "           & to_string(Q1_14ToReal(PrevXSync(1))) , DEBUG);
417         Log(MAGENTA & "PrevXSync(0): " & to_string(Q1_14ToReal(PrevXSync(0))) & ANSI_RESET, DEBUG);
418         Log(to_string(LF), DEBUG);
419         Log(          "PrevYSync(4): "           & to_string(Q1_14ToReal(PrevYSync(4))) , DEBUG);
420         Log(          "PrevYSync(3): "           & to_string(Q1_14ToReal(PrevYSync(3))) , DEBUG);
421         Log(          "PrevYSync(2): "           & to_string(Q1_14ToReal(PrevYSync(2))) , DEBUG);
422         Log(          "PrevYSync(1): "           & to_string(Q1_14ToReal(PrevYSync(1))) , DEBUG);
423         Log(MAGENTA & "PrevYSync(0): " & to_string(Q1_14ToReal(PrevYSync(0))) & ANSI_RESET, DEBUG);
424         Log(to_string(LF), DEBUG);
425
426         -- AffirmIf( AlertLogID : AlertLogIDType ; condition : boolean ; ReceivedMessage : string ;
427         --           ExpectedMessage : string ; Enable : boolean := FALSE) ;
428         AffirmIf (
429           CordicTBLogID,
430           Cond,
431
432           -- Received message
```

```
433           Color &
434           PrevFuncStringSync(LATENCY) &
435           " X: " & to_string(Q1_14ToReal(PrevXSync(LATENCY)), 6) &
436           " ("   & to_string(int_to_slv(PrevXSync(LATENCY), WORD_SIZE)) & ") " &
437           " Y: " & to_string(Q1_14ToReal(PrevYSync(LATENCY)), 6) & " " &
438           " ("   & to_string(int_to_slv(PrevYSync(LATENCY), WORD_SIZE)) & ") " &
439           HT & "Received: " & to_string (Q1_14ToReal(R_TB), 6) & " ( " & to_string(R_TB) & " ) " & ANSI_RESET,
440
441           -- Expected message
442           HT & Color & "PrevExpectedResult(" & to_string(LATENCY) & ") is " &
443           to_string(Q1_14ToReal(PrevExpectedResultSync(LATENCY)), 6) &
444           " (" & to_string(int_to_slv(PrevExpectedResultSync(LATENCY), WORD_SIZE)) & ")"
445           & ANSI_RESET &
446           " Diff: " & to_string(Diff)
447         );
448       end if;
449
450       -- We only want to print these messages once - the first time we see that they are covered.
451       if (IsCovered(CordicTrigCovID) and not TrigDone) then
452         Log(MAGENTA & "All trig tests finished." & ANSI_RESET, ALWAYS);
453         TrigDone := true;
454       end if;
455
456       if (IsCovered(CordicLinearCovID) and not LinearDone) then
457         Log(MAGENTA & "All linear tests finished." & ANSI_RESET, ALWAYS);
458         LinearDone := true;
459       end if;
460
461       if (IsCovered(CordicHyperCovID) and not HyperDone) then
462         Log(MAGENTA & "All hyperbolic tests finished." & ANSI_RESET, ALWAYS);
463         HyperDone := true;
464       end if;
465
466       if (IsCovered(CordicTrigCovID) and IsCovered(CordicLinearCovID) and
467           IsCovered(CordicHyperCovID))
468       then
469         Log("All tests finished.", ALWAYS);
470         ReportAlerts;
471         finish;
472       else
473
474         -- Test trig, linear, then hyperbolic.
475         if    (not IsCovered(CordicTrigCovID)) then
476
477           -- Loop until we have found a value that won't overflow
478           while (true) loop
479
480             -- Pick a random point. Note that we only set the Current values to these
481             -- values once we are sure they won't overflow.
482             (TmpFunc, TmpX) := GetRandPoint(CordicTrigCovID);
483
484             case uint_to_slv(TmpFunc, Func_TB'length) is
485
486               when F_COS_X    =>
```

```
487
488               if ( WillOverflow( X => Q1_14ToReal(TmpX),
489                                  Y => 0.0,
490                                  Func => uint_to_slv(TmpFunc, Func_TB'length)))
491               then
492                 Log(HT & "cos(" & to_string(Q1_14ToReal(TmpX)) & ")" &
493                          " will overflow, aborting test...", INFO);
494               else
495
496                 -- Store the prev values and set the current ones.
497                 PrevFuncSync(1 to 4) := PrevFuncSync(0 to 3);
498                 PrevFuncSync(0)      := CurrentFunc;
499                 CurrentFunc          := TmpFunc;
500
501                 PrevXSync(1 to 4)    := PrevXSync(0 to 3);
502                 PrevXSync(0)         := CurrentX;
503                 CurrentX             := TmpX;
504
505                 PrevYSync(1 to 4)    := PrevYSync(0 to 3);
506                 PrevYSync(0)         := CurrentY;
507
508                 PrevFuncStringSync(1 to 4) := PrevFuncStringSync(0 to 3);
509                 PrevFuncStringSync(0)      := CurrentFuncString;
510                 CurrentFuncString          := "cos(x) ";
511
512                 PrevExpectedResultSync(1 to 4) := PrevExpectedResultSync(0 to 3);
513                 PrevExpectedResultSync(0)      := ExpectedResult;
514
515                 -- Calculate the expected result.
516                 ExpectedResult  := RealToQ1_14( cos (Q1_14ToReal(CurrentX)));
517
518                 -- Set the inputs.
519                 Func_TB         <= uint_to_slv(CurrentFunc, Func_TB'length);
520                 X_TB            <= int_to_slv(CurrentX, WORD_SIZE);
521                 Y_TB            <= (others => '0');
522
523                 -- Mark the test as covered.
524                 ICover(CordicTrigCovID, (CurrentFunc, CurrentX));
525                 Log(CordicTBLogID, " (Trig is " & to_string(GetCov(CordicTrigCovID), 3) & "% covered) ",
                    INFO);
526
527                 -- Stop looping.
528                 exit;
529
530               end if; -- WillOverflow(...)
531
532             when F_SIN_X    =>
533
534               if ( WillOverflow( X => Q1_14ToReal(TmpX),
535                                  Y => 0.0,
536                                  Func => uint_to_slv(TmpFunc, Func_TB'length)))
537               then
538                 Log(HT & "sin(" & to_string(Q1_14ToReal(TmpX)) & ") " &
539                          "will overflow, aborting test...", INFO);
```

```
540               else
541
542                 -- Store the prev values and set the current ones.
543                 PrevFuncSync(1 to 2) := PrevFuncSync(0 to 1);
544                 PrevFuncSync(0)      := CurrentFunc;
545
546                 CurrentFunc          := TmpFunc;
547                 PrevXSync(1 to 2)    := PrevXSync(0 to 1);
548                 PrevXSync(0)         := CurrentX;
549
550                 PrevYSync(1 to 2)    := PrevYSync(0 to 1);
551                 PrevYSync(0)         := CurrentY;
552                 CurrentX             := TmpX;
553
554                 PrevFuncStringSync(1 to 4) := PrevFuncStringSync(0 to 3);
555                 PrevFuncStringSync(0)      := CurrentFuncString;
556                 CurrentFuncString          := "sin(x) ";
557
558                 PrevExpectedResultSync(1 to 4) := PrevExpectedResultSync(0 to 3);
559                 PrevExpectedResultSync(0)      := ExpectedResult;
560
561                 -- Calculate the expected result.
562                 ExpectedResult      := RealToQ1_14( sin (Q1_14ToReal(CurrentX)) );
563
564                 -- Set the inputs.
565                 Func_TB             <= uint_to_slv(CurrentFunc, Func_TB'length);
566                 X_TB                <= int_to_slv(CurrentX, WORD_SIZE);
567                 Y_TB                <= (others => '0');
568
569                 -- Mark the test as covered.
570                 ICover(CordicTrigCovID, (CurrentFunc, CurrentX));
571                 Log(CordicTBLogID, " (Trig is " & to_string(GetCov(CordicTrigCovID), 3) & "% covered) ",
                    INFO);
572                 -- Stop looping.
573                 exit;
574
575               end if; -- WillOverflow(...)
576
577               when others =>
578                 Alert("Unknown function " & to_string(uint_to_slv(TmpFunc, Func_TB'length)) &
579                     " in trig tests.", ERROR);
580             end case; -- uint_to_slv(TmpFunc, Func_TB'length)
581           end loop;  -- while (true)
582
583         -- Log Debug info.
584         Log(RepeatChar('-', 100), DEBUG);
585         PrintFunc(CurrentX, CurrentY, CurrentFunc);
586         Log("Current X: " & to_string(Q1_14ToReal(CurrentX), 4), DEBUG);
587         Log("Current Y: " & to_string(Q1_14ToReal(CurrentY), 4), DEBUG);
588         Log("Expected:  " & to_string(Q1_14ToReal(ExpectedResult), 4), DEBUG);
589         Log(RepeatChar('-', 100), DEBUG);
590
591
592       elsif (not IsCovered(CordicLinearCovID)) then
```

```
593
594         -- All trig tests done, test linear.
595
596         -- Loop until we have found a value that won't overflow.
597         while (true) loop
598
599           -- Pick a random point. Note that we only set the Current values to these
600           -- values once we are sure they won't overflow.
601           (TmpFunc, TmpX, TmpY) := GetRandPoint(CordicLinearCovID);
602
603           case uint_to_slv(TmpFunc, Func_TB'length) is
604
605             when F_X_MULT_Y =>
606
607               if (WillOverflow(X => Q1_14ToReal(TmpX),
608                                Y => Q1_14ToReal(TmpY),
609                                Func => uint_to_slv(TmpFunc, Func_TB'length)))
610               then
611                 Log(HT & "(" & to_string(Q1_14ToReal(TmpX)) & " * " &
612                                 to_string(Q1_14ToReal(TmpY)) & ")" &
613                          "will overflow, aborting test...", INFO);
614               else
615
616                 -- Store the prev values and set the current ones.
617                 PrevFuncSync(1 to 4) := PrevFuncSync(0 to 3);
618                 PrevFuncSync(0)      := CurrentFunc;
619                 CurrentFunc          := TmpFunc;
620
621                 PrevXSync(1 to 4)    := PrevXSync(0 to 3);
622                 PrevXSync(0)         := CurrentX;
623
624                 PrevYSync(1 to 4)    := PrevYSync(0 to 3);
625                 PrevYSync(0)         := CurrentY;
626
627                 CurrentX := TmpX;
628                 CurrentY := TmpY;
629
630                 PrevFuncStringSync(1 to 4) := PrevFuncStringSync(0 to 3);
631                 PrevFuncStringSync(0)      := CurrentFuncString;
632                 CurrentFuncString          := "x*y    ";
633
634                 PrevExpectedResultSync(1 to 4) := PrevExpectedResultSync(0 to 3);
635                 PrevExpectedResultSync(0)      := ExpectedResult;
636
637                 -- Calculate the expected result.
638                 ExpectedResult  := RealToQ1_14(Q1_14ToReal(CurrentX) * Q1_14ToReal(CurrentY));
639
640                 -- Set the inputs.
641                 Func_TB         <= uint_to_slv(CurrentFunc, Func_TB'length);
642                 X_TB            <= int_to_slv(CurrentX, WORD_SIZE);
643                 Y_TB            <= int_to_slv(CurrentY, WORD_SIZE);
644
645                 -- Mark the test as covered.
646                 ICover (CordicLinearCovID, (CurrentFunc, CurrentX, CurrentY));
```

```
647               Log(CordicTBLogID, " (Linear is " & to_string(GetCov(CordicLinearCovID), 3) &
648                                  "% covered) ", INFO);
649               exit;
650
651             end if; -- WillOverflow(...)
652
653
654           when F_Y_DIV_X  =>
655
656             if (WillOverflow(X => Q1_14ToReal(TmpX),
657                              Y => Q1_14ToReal(TmpY),
658                              Func => uint_to_slv(TmpFunc, Func_TB'length)))
659             then
660               Log(HT & "(" & to_string(Q1_14ToReal(TmpY)) & " / " &
661                              to_string(Q1_14ToReal(TmpX)) &
662                   ") will overflow, aborting test...", INFO);
663             else
664
665               -- Store the prev values and set the current ones.
666               PrevFuncSync(1 to 4) := PrevFuncSync(0 to 3);
667               PrevFuncSync(0)      := CurrentFunc;
668               CurrentFunc          := TmpFunc;
669
670               PrevXSync(1 to 4)    := PrevXSync(0 to 3);
671               PrevXSync(0)         := CurrentX;
672
673               PrevYSync(1 to 4)    := PrevYSync(0 to 3);
674               PrevYSync(0)         := CurrentY;
675
676               CurrentX := TmpX;
677               CurrentY := TmpY;
678
679               PrevFuncStringSync(1 to 4) := PrevFuncStringSync(0 to 3);
680               PrevFuncStringSync(0)      := CurrentFuncString;
681               CurrentFuncString          := "y/x    ";
682
683               PrevExpectedResultSync(1 to 4) := PrevExpectedResultSync(0 to 3);
684               PrevExpectedResultSync(0)      := ExpectedResult;
685
686               -- Calculate the expected result.
687               ExpectedResult := RealToQ1_14(Q1_14ToReal(CurrentY) / Q1_14ToReal(CurrentX));
688
689               -- Set the inputs.
690               Func_TB        <= uint_to_slv(CurrentFunc, Func_TB'length);
691               X_TB           <= int_to_slv(CurrentX, WORD_SIZE);
692               Y_TB           <= int_to_slv(CurrentY, WORD_SIZE);
693
694               -- Mark the test as covered.
695               ICover (CordicLinearCovID, (CurrentFunc, CurrentX, CurrentY));
696               Log(CordicTBLogID, " (Linear is " & to_string(GetCov(CordicLinearCovID), 3) &
697                                  "% covered) ", INFO);
698               exit;
699             end if; -- WillOverflow(...)
700
```

```
701             when others =>
702               Alert("Unknown function " & to_string(uint_to_slv(TmpFunc, Func_TB'length)) &
703                   " in linear tests.", ERROR);
704               stop(1);
705
706             end case;  -- uint_to_slv(TmpFunc)
707           end loop; -- while (true)
708
709           Log(RepeatChar('-', 100), DEBUG);
710           PrintFunc(CurrentX, CurrentY, CurrentFunc);
711           Log("Current X: " & to_string(Q1_14ToReal(CurrentX), 4), DEBUG);
712           Log("Current Y: " & to_string(Q1_14ToReal(CurrentY), 4), DEBUG);
713           Log("Expected:  " & to_string(Q1_14ToReal(ExpectedResult), 4), DEBUG);
714           Log(RepeatChar('-', 100), DEBUG);
715
716
717         elsif (not IsCovered(CordicHyperCovID)) then
718           -- All trig and linear tests done, test hyperbolic.
719
720           -- Loop until we have found a value that won't overflow
721           while (true) loop
722
723             -- Pick a random point. Note that we only set the Current values to these
724             -- values once we are sure they won't overflow.
725             (TmpFunc, TmpX) := GetRandPoint(CordicHyperCovID);
726
727             case uint_to_slv(TmpFunc, Func_TB'length) is
728
729               when F_COSH_X   =>
730
731                 if ( WillOverflow( X    => Q1_14ToReal(TmpX),
732                                    Y    => 0.0,
733                                    Func => uint_to_slv(TmpFunc, Func_TB'length)))
734                 then
735                   Log(HT & "cosh(" & to_string(Q1_14ToReal(TmpX)) & ") will overflow, aborting test...",
                      INFO);
736                 else
737
738                   -- Store the prev values and set the current ones.
739                   PrevFuncSync(1 to 4) := PrevFuncSync(0 to 3);
740                   PrevFuncSync(0)      := CurrentFunc;
741                   CurrentFunc          := TmpFunc;
742
743                   PrevXSync(1 to 4)    := PrevXSync(0 to 3);
744                   PrevXSync(0)         := CurrentX;
745                   CurrentX             := TmpX;
746
747                   PrevYSync(1 to 4)    := PrevYSync(0 to 3);
748                   PrevYSync(0)         := CurrentY;
749
750                   PrevFuncStringSync(1 to 4) := PrevFuncStringSync(0 to 3);
751                   PrevFuncStringSync(0)      := CurrentFuncString;
752                   CurrentFuncString          := "cosh(x)";
753
```

```
754                 PrevExpectedResultSync(1 to 4) := PrevExpectedResultSync(0 to 3);
755                 PrevExpectedResultSync(0)      := ExpectedResult;
756
757                 -- Calculate the expected result.
758                 ExpectedResult      := RealToQ1_14( cosh (Q1_14ToReal(CurrentX)));
759
760                 -- Set the inputs.
761                 Func_TB             <= uint_to_slv(CurrentFunc, Func_TB'length);
762                 X_TB                <= int_to_slv(CurrentX, WORD_SIZE);
763                 Y_TB                <= (others => '0');
764
765                 -- Mark the test as covered.
766                 ICover(CordicHyperCovID, (CurrentFunc, CurrentX));
767                 Log(CordicTBLogID, " (Hyperbolic is " & to_string(GetCov(CordicHyperCovID), 3) &
768                                    "% covered) ", INFO);
769                 exit;
770               end if; -- WillOverflow(...)
771
772             when F_SINH_X   =>
773
774               if ( WillOverflow( X    => Q1_14ToReal(TmpX),
775                                  Y    => 0.0,
776                                  Func => uint_to_slv(TmpFunc, Func_TB'length)))
777               then
778                 Log(HT & "sinh(" & to_string(Q1_14ToReal(TmpX)) & ") will overflow, aborting test...",
                    INFO);
779               else
780
781                 -- Store the prev values and set the current ones.
782                 PrevFuncSync(1 to 4) := PrevFuncSync(0 to 3);
783                 PrevFuncSync(0)      := CurrentFunc;
784                 CurrentFunc          := TmpFunc;
785
786                 PrevXSync(1 to 4)    := PrevXSync(0 to 3);
787                 PrevXSync(0)         := CurrentX;
788                 CurrentX             := TmpX;
789
790                 PrevYSync(1 to 4)    := PrevYSync(0 to 3);
791                 PrevYSync(0)         := CurrentY;
792
793                 PrevFuncStringSync(1 to 4) := PrevFuncStringSync(0 to 3);
794                 PrevFuncStringSync(0)      := CurrentFuncString;
795                 CurrentFuncString          := "sinh(x)";
796
797                 PrevExpectedResultSync(1 to 4) := PrevExpectedResultSync(0 to 3);
798                 PrevExpectedResultSync(0)      := ExpectedResult;
799
800                 -- Calculate the expected result.
801                 ExpectedResult      := RealToQ1_14( sinh (Q1_14ToReal(CurrentX)));
802
803                 -- Set the inputs.
804                 Func_TB             <= uint_to_slv(CurrentFunc, Func_TB'length);
805                 X_TB                <= int_to_slv(CurrentX, WORD_SIZE);
806                 Y_TB                <= (others => '0');
```

```
807
808                 -- Mark the test as covered.
809                 ICover(CordicHyperCovID, (CurrentFunc, CurrentX));
810                 Log(CordicTBLogID, " (Hyperbolic is " & to_string(GetCov(CordicHyperCovID), 3) &
811                                    "% covered) ", INFO);
812                 exit;
813               end if; -- WillOverflow(...)
814
815             when others =>
816               Alert("Unknown function " & to_string(uint_to_slv(TmpFunc, Func_TB'length)) &
817                   " in hyperbolic tests.", ERROR);
818               stop(1);
819
820           end case;  -- uint_to_slv(TmpFunc, Func_TB'length)
821         end loop; -- while (true)
822
823         Log(RepeatChar('-', 100), DEBUG);
824         PrintFunc(CurrentX, CurrentY, CurrentFunc);
825         Log("Current X: " & to_string(Q1_14ToReal(CurrentX), 4), DEBUG);
826         Log("Current Y: " & to_string(Q1_14ToReal(CurrentY), 4), DEBUG);
827         Log("Expected:  " & to_string(Q1_14ToReal(ExpectedResult), 4), DEBUG);
828         Log(RepeatChar('-', 100), DEBUG);
829
830       end if; -- (not IsCovered(CordicTrigCovID))
831     end if; -- IsCovered (<all three CovIDs>)
832   end if; -- rising_edge(Clk)
833 end process; -- RunTest
834
835 end TestBench;
836
```